Classificação do Nível de Stress

Machine Learning 2026 – LIACD
Departamento de Engenharia Informática

Duarte Prazeres
duartecastromarques@gmail.com
Nº 2023214962

Marília Brito
mariliamontemorb@gmail.com
Nº 2024225969

Maio 2026

Índice

1 Introdução **3**

2 Background **3**
2.1 Trabalhos na Plataforma Kaggle 3
2.2 Literatura Académica Relacionada 4
2.3 Posicionamento deste Trabalho 4

3 Dataset e Análise Exploratória **5**
3.1 Descrição do Dataset 5
3.2 Limpeza e Qualidade dos Dados 5
3.3 Visualizações EDA 5

4 Metodologia **9**
4.1 Transformação do Target 9
4.2 Feature Engineering 10
4.3 Pré-processamento 11
4.3.1 Divisão dos Dados 11
4.3.2 Normalização e Prevenção de Data Leakage 12
4.3.3 Lidar com o Desequilíbrio de Classes 12
4.4 Métricas de Avaliação 12
4.5 Modelos Implementados 13
4.5.1 Support Vector Machine (SVM) 13
4.5.2 Rede Neuronal (MLPClassifier) 13
4.5.3 XGBoost – 3.º Modelo 14
4.6 Validação Cruzada 14

5 Resultados e Discussão **15**
5.1 Resultados no Conjunto de Teste (Hold-out) 15
5.2 Matrizes de Confusão 16
5.2.1 Cenário A – Classificação Binária 16
5.2.2 Cenário B – 5 Classes 17
5.2.3 Cenário C – 10 Classes 17
5.3 Curvas de Loss – Rede Neuronal 18
5.4 Análise Crítica por Cenário 18
5.4.1 Cenário A – Classificação Binária 18
5.4.2 Cenário B – 5 Classes 19
5.4.3 Cenário C – Trade-offs de Granularidade (10 Classes) 19
5.5 Comparação entre Modelos 20
5.6 Efeito do Tuning de Hiperparâmetros 20
5.7 Interpretabilidade Local (SHAP) 20
5.8 Testes de Significância Estatística 21
5.9 Limitações e Desafios 22

6 Conclusões **22**

1 Introdução

Neste trabalho, propusemo-nos a construir e avaliar um sistema completo de classificação do nível de stress com base em dados comportamentais recolhidos através de smartphones: utilização do ecrã, padrões de sono e atividade física. Para isso, utilizámos o dataset *Sleep, Screen Time and Stress Analysis*, disponível no Kaggle com 15 000 registos sintéticos [6].

Desde o início percebemos que este problema podia ser enquadrado de formas muito diferentes consoante a granularidade de classes que se pretendesse. Em vez de nos limitarmos a um único formato, decidimos atacá-lo sob três perspetivas de complexidade crescente:

- **Cenário A** – Classificação binária: *sem stress* (0) vs *com stress* (1);
- **Cenário B** – 5 classes: Sem stress, Baixo, Moderado, Alto, Extremo;
- **Cenário C** – 10 classes: escala granular Nível 0 a Nível 9.

Esta estrutura em três cenários permitiu-nos não só avaliar o desempenho dos modelos, mas também estudar como a complexidade do problema de classificação afeta os resultados – algo que raramente encontrámos abordado de forma sistemática na literatura.

Para cobrir diferentes paradigmas de aprendizagem automática, escolhemos três modelos: **SVM**, **Rede Neuronal (MLP)** e **XGBoost**. Avaliámo-los com validação cruzada estratificada, otimização sistemática de hiperparâmetros e testes de significância estatística, para garantir que as conclusões fossem robustas e não produto do acaso. Os principais resultados que obtivemos foram F1 macro de $\approx$0.95 (Cenário A), $\approx$0.82 (Cenário B) e $\approx$0.58 (Cenário C).

2 Background

Antes de começarmos a modelar, considerámos importante perceber o que outros investigadores e a comunidade Kaggle já tinham feito sobre este problema ou problemas semelhantes. Queríamos dois tipos de informação: referências metodológicas que nos ajudassem a tomar decisões mais informadas, e resultados de comparação que nos permitissem avaliar a qualidade do que iriamos produzir.

2.1 Trabalhos na Plataforma Kaggle

Para contextualizar e comparar o nosso trabalho, começámos por explorar a secção *Code* do próprio dataset no Kaggle, onde a comunidade publica notebooks com análises independentes sobre os mesmos dados.

O trabalho de Badrnezhad [7] foi o primeiro que analisámos em detalhe, por desenvolver um pipeline ETL completo sobre um dataset estruturalmente idêntico ao nosso. O que nos pareceu mais relevante é que este trabalho se ficou pela limpeza de dados e análise exploratória, sem avançar para modelação preditiva – o que nos motivou a ir mais longe e a perceber que havia espaço para construir e avaliar modelos de classificação robustos sobre estes dados.

Analisámos igualmente o trabalho de Tolstoyj [8], que utilizou *Random Forest* em modo de **regressão** sobre dados sintéticos do mesmo tipo (sono, tempo de ecrã, stress). A conclusão reportada foi que "as correlações iniciais entre as variáveis comportamentais

e o nível de stress são fracas", o que nos levou a questionar se a fraqueza do sinal se devia à abordagem de regressão ou ao dataset em si. Decidimos então adotar uma estratégia de **classificação** com discretização explícita da variável-alvo em vez de regressão direta – e os resultados que obtivemos confirmaram que esta foi a decisão correta: com F1 macro de ≈0.95 no Cenário A, o sinal preditivo é claramente mais rico do que o sugerido por uma abordagem regressiva.

2.2 Literatura Académica Relacionada

Além dos trabalhos na plataforma Kaggle, decidimos ir mais longe e fazer uma pesquisa de papers e obras na literatura académica para perceber o estado da arte em problemas semelhantes. Contudo, esta tarefa é difícil e demorada pois exige muito tempo a investigar ao pormenor imensos trabalhos de variadas fontes. Por isso, para otimizar o nosso tempo e para concentrar mais esforços nas tarefas seguintes do trabalho, decidimos recorrer a um agente de inteligência artificial corrido sobre o modelo Sonnet 4.7 da Claude (Anthropic) para nos ajudou a encontrar e explicar os principais trabalhos relacionados com o nosso problema:

Classificação de distúrbios do sono e stress: Encontrámos um estudo de Huang et al. [9] que nos pareceu particularmente relevante por ter utilizado features muito sobreponíveis às nossas (duração do sono, qualidade, nível de stress, IMC). Os autores compararam seis algoritmos – Random Forest, Árvore de Decisão, SVM, KNN, XGBoost e ANN – e concluíram que o **XGBoost obteve a melhor accuracy de 90.66%**. Esta descoberta pesou na nossa decisão de incluir o XGBoost como terceiro modelo obrigatório, e os nossos resultados acabaram por confirmar o seu valor.

Previsão de stress em padrões de sono: Deparámo-nos também com o trabalho de Lei et al. [10], que usaram Random Forest para classificação binária de stress a partir de dados de sono, reportando um F1-Score de 98%. O valor é superior ao que obtivemos (≈0.95), mas percebemos que isso se deve à maior simplicidade do dataset deles – com padrões de sono mais diferenciados e sem o ruído adicional das variáveis de tempo de ecrã. Esta comparação ajudou-nos a calibrar as expectativas e a perceber que os nossos resultados são, de facto, competitivos dado o contexto mais exigente do dataset que utilizámos.

Classificação de stress com sinais fisiológicos: Por curiosidade, analisámos também um trabalho com dados de sensores biométricos IoMT recolhidos durante o sono [11], onde todos os classificadores testados superaram 0.98 de accuracy. Percebemos rapidamente que este contexto é fundamentalmente diferente do nosso: sinais fisiológicos contínuos são muito mais discriminativos do que dados comportamentais sintéticos, pelo que não seria realista esperar resultados semelhantes. Esta distinção foi útil para enquadrar as limitações do nosso trabalho.

2.3 Posicionamento deste Trabalho

Depois de consolidar esta revisão, ficou-nos claro onde o nosso trabalho se posiciona. A Tabela 1 resume as principais diferenças e compara os resultados. Identificámos também um gap claro na literatura: nenhum dos trabalhos que analisámos aborda a classificação de stress com 5 ou 10 classes de forma sistemática – todos se focam na classificação binária. Esta lacuna constituiu uma motivação adicional para manter os três cenários no nosso projeto, tornando-o uma contribuição mais diferenciada.

Tabela 1: Comparação com trabalhos relacionados (classificação binária).

Trabalho	**Dados**	**Melhor Modelo**	**Acc.**	**F1**
Lei et al. [10]	Kaggle (sono sintético)	Random Forest	–	0.98
Huang et al. [9]	Kaggle (saúde/sono)	XGBoost	90.66%	–
Acosta et al. [11]	IoMT biosensores	Ensemble	>98%	–
Este trabalho (Cen. A)	**Kaggle (sintét.)**	**NN/MLP**	**95.02%**	**0.9502**

Os resultados que alcançámos para o Cenário A (F1 ≈ 0.95) são comparáveis à literatura para dados sintéticos comportamentais, com uma metodologia mais abrangente do que a maioria dos trabalhos consultados (3 modelos, 3 cenários, testes estatísticos).

3 Dataset e Análise Exploratória

3.1 Descrição do Dataset

Quando abrimos o dataset pela primeira vez, verificámos que continha 15 000 instâncias com 13 variáveis descrevendo o comportamento diário de diferentes utilizadores: tempo de ecrã diário, duração e qualidade do sono, atividade física, consumo de cafeína, *score* de fadiga mental e dados demográficos (idade, género, ocupação). A variável alvo, `stress_level`, é contínua – o que significou logo à partida que teríamos de a transformar em categorias antes de poder treinar qualquer modelo de classificação.

3.2 Limpeza e Qualidade dos Dados

Antes de avançarmos para qualquer análise, fizemos uma verificação cuidadosa da qualidade dos dados. O resultado foi encorajador:

- **Valores nulos**: não encontrámos nenhum em nenhuma coluna, pelo que não foi necessário imputar nem remover registos;
- **Duplicados**: não detetámos nenhum registo duplicado;
- **Colunas irrelevantes**: identificámos a coluna `user_id` como um identificador sem valor preditivo e excluímo-la antes de qualquer processamento.

No fundo, o dataset estava bastante limpo, o que nos permitiu focar o esforço nas etapas mais interessantes do projeto: a engenharia de features e a modelação.

3.3 Visualizações EDA

Para compreender os dados antes de os modelar, construímos um conjunto de visualizações exploratórias com um objetivo concreto: responder a três perguntas fundamentais – como se distribuem as variáveis individualmente? quais estão mais correlacionadas com o stress? e as classes que criámos estão razoavelmente equilibradas?

Começámos pelas distribuições univariadas de cada variável numérica (Figura abaixo), que nos deram uma primeira ideia da forma geral dos dados e de eventuais assimetrias.

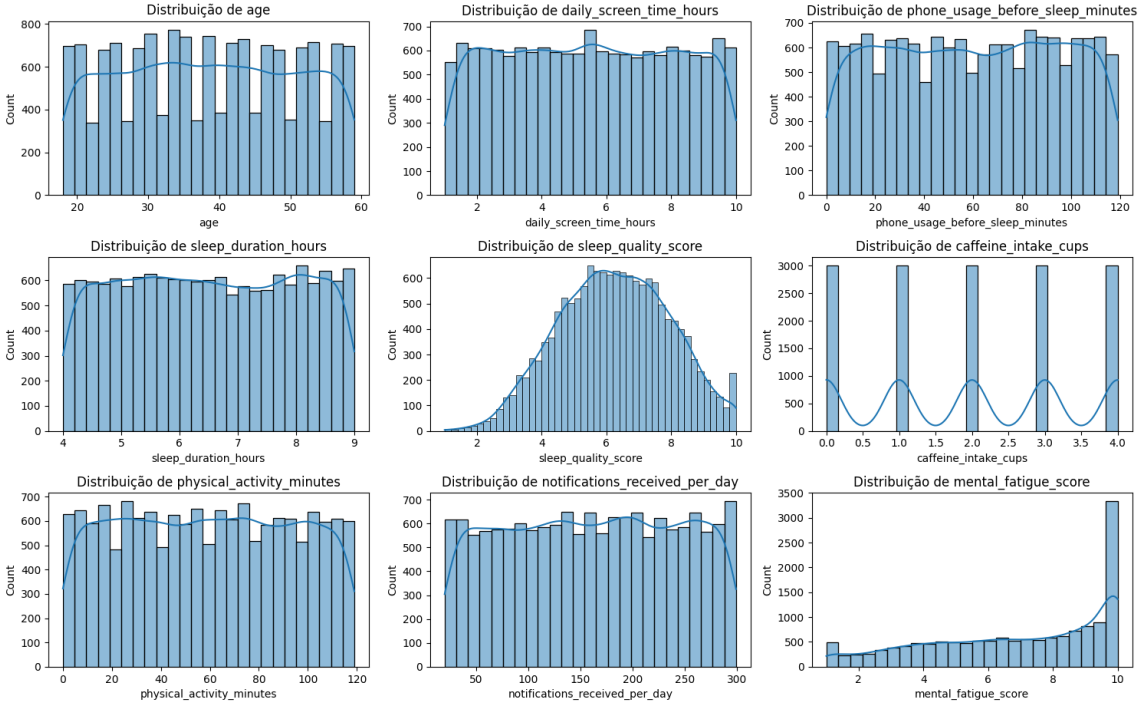


Figura 1: Distribuições univariadas das variáveis numéricas.

Ao analisar os histogramas, verificámos que a maioria das variáveis numéricas apresenta distribuições aproximadamente normais ou uniformes – comportamento esperado num dataset sintético. A ausência de assimetrias extremas e de outliers severos confirmou-nos que não seria necessária qualquer transformação logarítmica ou remoção de anomalias. Notámos que o `mental_fatigue_score` apresenta uma distribuição mais concentrada e simétrica, enquanto variáveis como `physical_activity_minutes` e `caffeine_intake_cups` mostram maior dispersão – detalhe que se revelaria relevante na análise de correlação.

De seguida, construímos dois heatmaps de correlação complementares: um de **Pearson** e outro de **Spearman**. A razão para usarmos as duas métricas foi metodológica: a correlação de Pearson mede apenas relações *lineares*, ao passo que a correlação de Spearman – por operar sobre os *ranks* dos valores em vez dos valores brutos – captura qualquer relação **monotónica**, seja ela logarítmica, exponencial ou de potência. Comparar as duas permite-nos perceber quais as relações têm uma componente não-linear significativa.

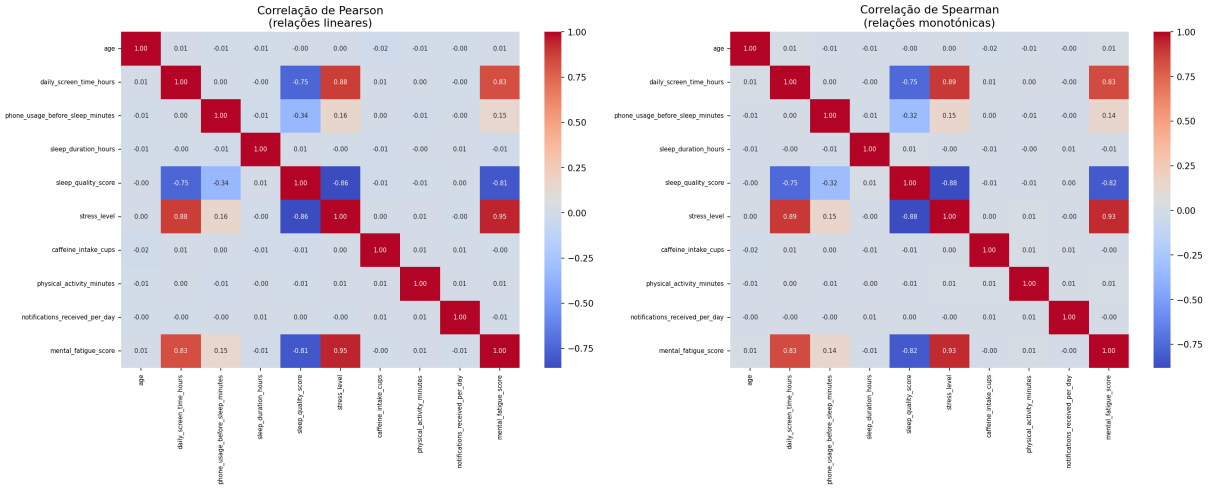


Figura 2: Heatmaps de correlação de Pearson (esquerda) e Spearman (direita). Os três preditores dominantes são os mesmos em ambas as métricas, mas `sleep_quality_score` e `daily_screen_time_hours` mostram correlação de Spearman ligeiramente superior.

A comparação dos dois heatmaps revelou dois resultados que consideramos importantes. Em primeiro lugar, os três preditores dominantes são os mesmos em ambas as métricas, com valores muito próximos entre si:

- `mental_fatigue_score`: Pearson $r = 0.95$, Spearman $\rho = 0.93$;
- `daily_screen_time_hours`: Pearson $r = 0.88$, Spearman $\rho = 0.89$;
- `sleep_quality_score`: Pearson $r = 0.86$, Spearman $\rho = 0.88$.

Em segundo lugar, e mais interessante, `daily_screen_time_hours` e `sleep_quality_score` apresentam correlação de Spearman *ligeiramente superior* à de Pearson – sinal de que estas relações têm uma componente monotónica não-linear que a Pearson não captura completamente. Pelo contrário, para o `mental_fatigue_score` a diferença é quase nula, sugerindo uma relação predominantemente linear com o stress. Surpreendeu-nos também verificar que variáveis como a atividade física, a duração do sono e o consumo de cafeína apresentam correlações próximas de zero em ambas as métricas – confirmando que o sinal preditivo do dataset se concentra nas três variáveis-chave acima, algo que influenciou diretamente a nossa estratégia de feature engineering.

Verificámos também a distribuição dos targets que criámos para cada cenário, para garantir que o processo de discretização não gerou classes extremamente desequilibradas.

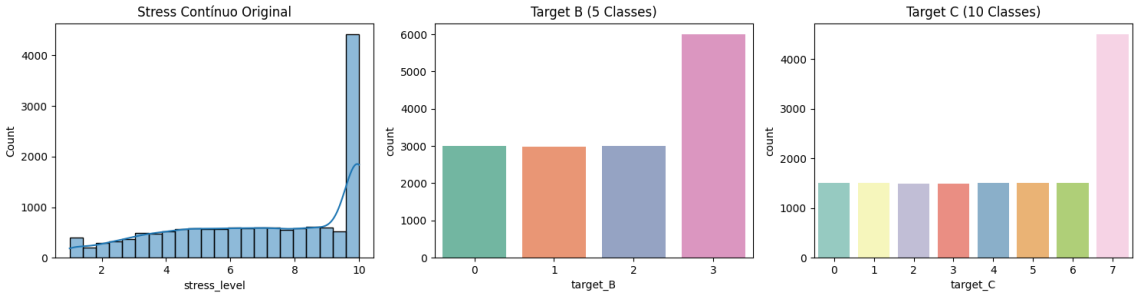


Figura 3: Distribuição dos targets nos Cenários A, B e C após discretização.

O gráfico confirma que o *equal-frequency binning* atingiu o objetivo pretendido: as classes estão aproximadamente equilibradas em todos os cenários – 50/50 no Cenário A, ≈20% por classe no Cenário B e ≈10% por classe no Cenário C. Este equilíbrio é fundamental para que o F1 macro seja uma métrica justa: com classes desequilibradas, um modelo que ignorasse as classes minoritárias ainda assim obteria accuracy elevada, tornando a avaliação enganadora. Ao garantir o equilíbrio à partida, evitámos este problema e tornámos a comparação entre cenários mais honesta.

Complementámos com um box plot da variável mais correlacionada por classe no Cenário B, que confirmou visualmente que as classes têm distribuições diferentes e que, portanto, o problema de classificação tem sinal preditivo real.

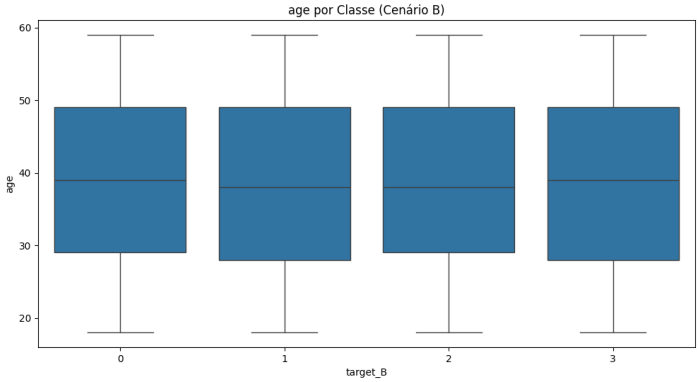


Figura 4: Box plot da variável mais correlacionada por classe (Cenário B).

O box plot confirma aquilo que a análise de correlação já antecipava: há uma separação clara e progressiva das medianas da variável mais correlacionada ao longo das 5 classes do Cenário B. As classes extremas (0 e 4) têm distribuições bem separadas com sobreposição mínima entre si, enquanto as classes intermédias (1, 2 e 3) apresentam intervalos interquartis com maior sobreposição mútua. Esta observação antecipou um padrão que depois confirmámos nas matrizes de confusão: os modelos distinguem bem os extremos, mas têm maior dificuldade nas classes intermédias – o que é semanticamente coerente, pois a diferença comportamental entre stress "baixo" e "moderado" é genuinamente subtil.

Por fim, gerámos um pairplot das três features mais correlacionadas com o stress para visualizar as relações entre pares de variáveis e avaliar a separabilidade visual das classes.

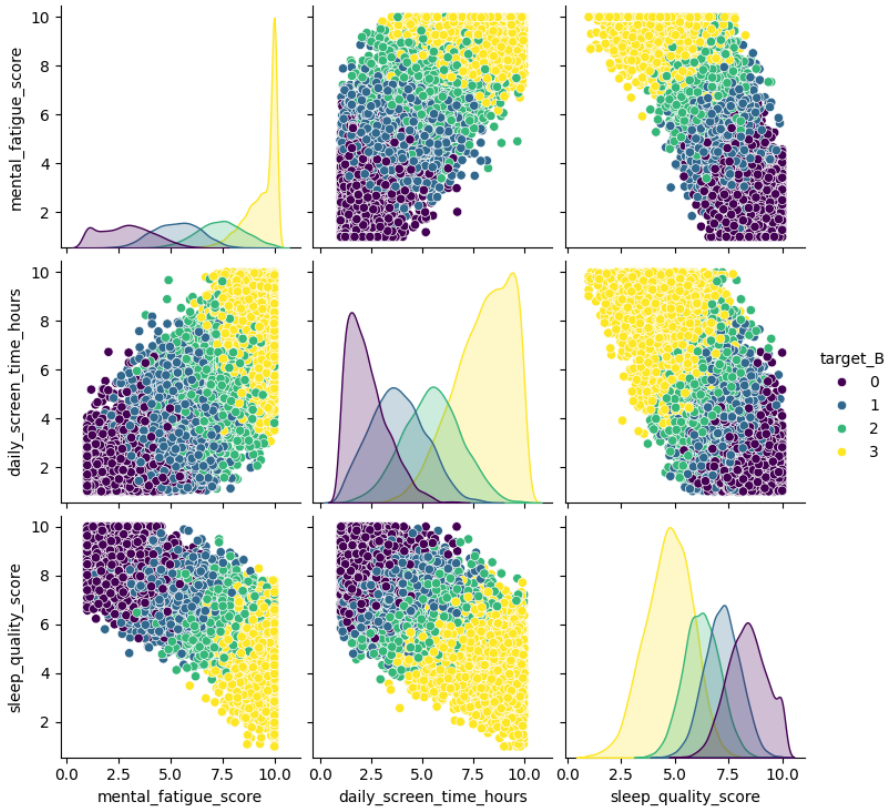


Figura 5: Pairplot das três features mais correlacionadas com o stress, coloridas por classe do Cenário B.

O pairplot confirma a estrutura do problema de forma visual. Nos scatter plots fora da diagonal, é percetível uma separação razoável entre as classes, especialmente nas projeções que envolvem o `mental_fatigue_score`: os pontos de classes extremas (coloração mais escura vs. mais clara) tendem a ocupar regiões distintas do espaço de features. Nas distribuições diagonais (KDE por classe), as classes intermédias sobrepõem-se significativamente entre si, mas distinguem-se das extremas. Ficámos satisfeitos com este resultado: a separabilidade visual, ainda que imperfeita, confirmou-nos que os modelos teriam sinal suficiente para aprender – e ajudou-nos a calibrar as expectativas de desempenho antes de avançar para a modelação.

4 Metodologia

4.1 Transformação do Target

Uma das primeiras decisões que tivemos de tomar foi como transformar a variável contínua `stress_level` em classes discretas para cada cenário. Não existe uma única forma correta de o fazer, por isso avaliámos diferentes abordagens e justificámos cada escolha:

- **Cenário A (2 Classes)**: Optámos por *equal-frequency binning* com ponto de corte na mediana (7.38), obtendo duas classes equilibradas "Sem Stress" (0) e "Com Stress" (1). A razão principal foi o equilíbrio resultante (≈50/50): com classes balanceadas, os modelos não ficam tendenciosos para a classe maioritária, o que simplifica a avaliação.

- **Cenário B (5 Classes)**: Escolhemos **Equal-Frequency Binning** em 5 quantis (Sem stress, Baixo, Moderado, Alto, Extremo). A lógica foi a mesma: ao garantir que cada classe tem exatamente o mesmo número de instâncias, evitámos classes esparsas que tornariam a aprendizagem injusta para os modelos.
- **Cenário C (10 Classes)**: Aqui experimentámos e comparámos explicitamente três métodos – Equal-Width, Equal-Frequency e K-Means – antes de decidir. Acabámos por escolher **Equal-Frequency** pelos mesmos motivos do Cenário B. Reconhecemos, porém, a principal fraqueza desta escolha: a sobreposição semântica entre classes adjacentes (a diferença de stress entre nível 4 e nível 5 é muito pequena) cria fronteiras artificiais que dificultam a tarefa dos modelos.

4.2 Feature Engineering

Olhando para as features originais do dataset, sentimos que podíamos enriquecer o espaço de atributos com variáveis que capturassem relações que as features brutas não expressavam diretamente. Criámos cinco novas variáveis, cada uma com uma justificação específica:

Feature	**Descrição e Justificação**
`ratio_usage_sleep`	Rácio ecrã/sono. Criámos esta feature porque acreditamos que o efeito do ecrã no stress não é absoluto, mas relativo à quantidade de sono – quem dorme pouco e usa muito o ecrã está em situação muito diferente de quem dorme bastante.
`ratio_activity_usage`	Atividade física dividida pelo tempo de ecrã. Representa um indicador de estilo de vida equilibrado: valores altos sugerem que a pessoa equilibra o sedentarismo digital com movimento.
`interaction_usage_fatigue`	Produto de `screen_time` × `mental_fatigue_score`. Criámos esta interação para capturar o efeito multiplicativo entre os dois preditores mais fortes. Acabou por ser **removida** porque apresentava correlação > 0.95 com `mental_fatigue_score`, o que introduziria multicolinearidade sem acrescentar informação nova.
`dorme_menos_6h`	Flag binária: sono $<$ 6h. Adicionámos este indicador clínico porque privação severa de sono é um fator de risco bem documentado para o stress, e uma variável binária captura este limiar de forma mais expressiva do que o valor contínuo.
`sedentario`	Flag binária: atividade $<$ 30min. Pelo mesmo raciocínio, criámos um indicador explícito de sedentarismo, que pode ser um fator de amplificação do stress não facilmente capturado pela variável contínua de atividade.

Para validar as nossas escolhas, corremos uma avaliação de importância preliminar com um Random Forest simples e confirmámos que as features com correlação cruzada > 0.95 seriam eliminadas automaticamente. A Figura 6 confirma que o `mental_fatigue_score` e o `daily_screen_time_hours` continuam a ser os preditores dominantes – o que nos deu confiança de que o dataset tem sinal preditivo real e de que as nossas features adicionais contribuem, ainda que marginalmente.

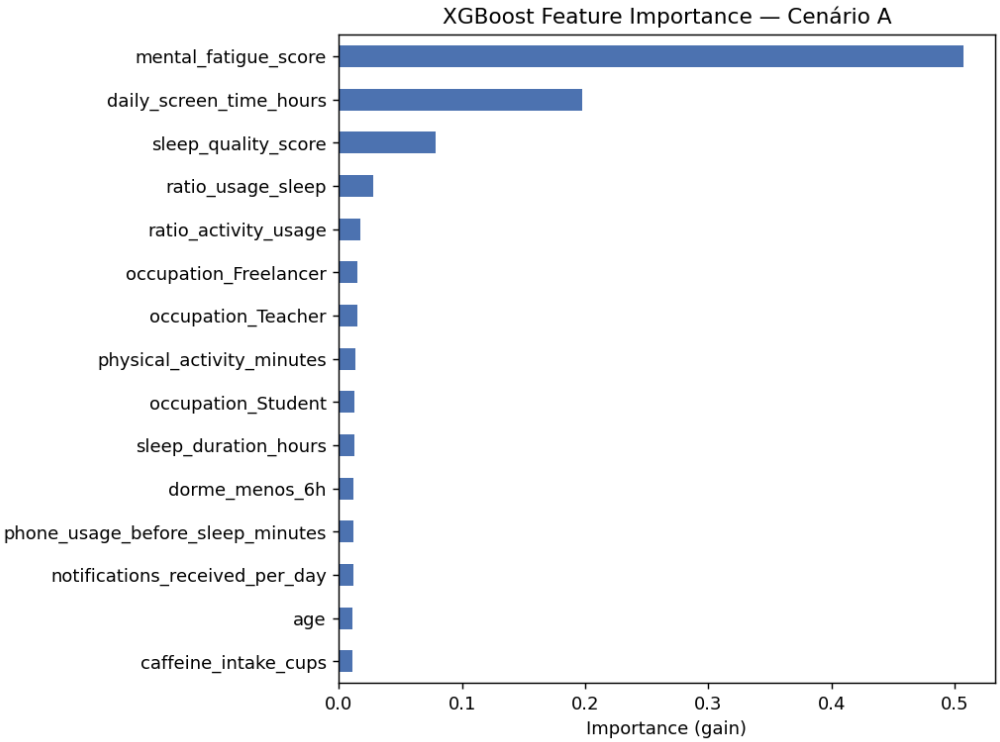


Figura 6: Importância das features calculada pelo XGBoost (Cenário A) via Gini impurity.

O gráfico de importância confirma e quantifica o que a análise de correlação já sugeria. O `mental_fatigue_score` domina de forma clara, com uma importância substancialmente acima de todas as outras features. O `daily_screen_time_hours` e o `sleep_quality_score` surgem em segundo e terceiro lugar, respetivamente. Mais encorajador ainda, os rácios que criámos – `ratio_usage_sleep` e `ratio_activity_usage` – aparecem com importância não nula, validando a nossa decisão de os criar. As features com importância próxima de zero, como `caffeine_intake_cups` ou `age`, confirmam o que as correlações de Pearson e Spearman já tinham revelado: neste dataset sintético, o sinal preditivo concentra-se em três variáveis-chave, sendo as restantes maioritariamente ruidosas.

4.3 Pré-processamento

4.3.1 Divisão dos Dados

Para avaliar os modelos de forma rigorosa, dividimos os dados em três conjuntos distintos, seguindo uma proporção **70/15/15** (treino/validação/teste):

- Treino: 10 500 instâncias – usadas para ajustar os parâmetros dos modelos;
- Validação: 2 250 instâncias – usadas durante a otimização de hiperparâmetros;

- Teste: 2 250 instâncias – reservadas e intocadas até à avaliação final.

Em todas as divisões usámos o parâmetro `stratify`, que garante que a proporção de classes é mantida em cada conjunto – decisão fundamental para que a validação cruzada seja justa e representativa.

4.3.2 Normalização e Prevenção de Data Leakage

Decidimos normalizar todas as features com `StandardScaler`, porque modelos como o SVM são muito sensíveis à escala das variáveis e a sua performance degrada drasticamente sem normalização. O passo crítico foi garantir que o scaler era *exclusivamente* calculado sobre os dados de treino (`fit_transform`) e apenas aplicado – sem recalcular – nos conjuntos de validação e teste (`transform`). Fizemo-lo assim para evitar *data leakage*: se calculássemos a média e o desvio padrão com base em todos os dados incluindo o teste, estaríamos a dar ao modelo informação sobre dados que ainda não deveria ter visto, inflacionando artificialmente a performance. Para reforçar esta proteção durante a validação cruzada, encapsulámos o scaler num `Pipeline` do scikit-learn, que repete automaticamente este processo correto em cada fold.

4.3.3 Lidar com o Desequilíbrio de Classes

Nos Cenários B e C, confrontámo-nos com o problema do desequilíbrio entre classes. Experimentámos três abordagens diferentes antes de tomar uma decisão:

- **SMOTE e ADASYN (Oversampling)**: geram instâncias sintéticas para aumentar as classes minoritárias – testámos, mas optámos por não usar porque alteram a distribuição original dos dados;
- **RandomUnderSampler**: elimina instâncias das classes maioritárias – descartámos porque resulta em perda de informação relevante;
- **Ponderação interna dos modelos**: a abordagem que acabámos por escolher. Usámos `class_weight` e `sample_weight` com o valor `'balanced'` no SVM e XGBoost respetivamente, penalizando mais os erros nas classes sub-representadas sem alterar os dados originais.

A nossa preferência pela ponderação interna deveu-se precisamente a esta vantagem: preserva a distribuição real dos dados, tornando a avaliação mais honesta.

4.4 Métricas de Avaliação

Tivemos de pensar cuidadosamente em quais as métricas mais adequadas para este problema. Desde logo, percebemos que usar apenas **accuracy** seria enganador com classes desbalanceadas, porque um modelo que preveja sempre a classe maioritária pode ter accuracy elevada mesmo sendo inútil. Por isso definimos o seguinte conjunto de métricas:

Cenário	Métrica Principal	Métricas Complementares
A (Binário)	F1-Score (macro)	Precision, Recall, ROC-AUC, MCC
B (5 Classes)	F1-Score (macro)	Weighted F1, Cohen's Kappa

C (10 Classes)	F1-Score (macro)	Top-2 Accuracy, Weighted F1, Cohen's Kappa

Escolhemos o **F1-Score macro** como métrica principal porque trata todas as classes de forma igualitária, independentemente da sua frequência – penalizando os modelos que ignoram classes minoritárias. O **Cohen's Kappa** complementa ao medir a concordância acima do que seria esperado por acaso. O **MCC** é particularmente robusto a desequilíbrios. Para o Cenário C, acrescentámos ainda a **Top-2 Accuracy**, que considera uma previsão correta se a classe verdadeira estiver entre as duas mais prováveis – útil num problema de 10 classes onde o erro de um nível adjacente tem pouco significado prático.

4.5 Modelos Implementados

Para a escolha dos três modelos, o enunciado tornava o SVM e a Rede Neuronal obrigatórios. Para o terceiro, tivemos de justificar a nossa opção. Depois de pesquisar a literatura – em particular o trabalho de Huang et al. [9] que mostrou o XGBoost como melhor algoritmo num problema semelhante – optámos pelo XGBoost. A seguir descrevemos as nossas decisões de implementação para cada modelo.

4.5.1 Support Vector Machine (SVM)

O SVM [1] foi incluído por obrigação, mas também por convicção: trata-se de um modelo com fortes garantias teóricas (minimização do risco estrutural) que se adequa bem ao nosso problema. A razão principal é que os limites entre classes comportamentais como "stress moderado" e "stress alto" são intrinsecamente ruidosos e difusos – e a maximização da margem do SVM confere exactamente o tipo de robustez necessária nestes casos. Além disso, o kernel RBF permite capturar as interações não-lineares entre sono, ecrã e atividade física sem requerer que as criemos manualmente.

Implementação:

- Encapsulámos o modelo num `Pipeline([StandardScaler, SVC])` para prevenir data leakage durante a validação cruzada;
- Kernel RBF: $K(x, x') = e^{-\gamma \|x - x'\|^2}$;
- `class_weight='balanced'` e `probability=True` (necessário para ROC-AUC);
- Para o tuning, usámos **GridSearchCV** com apenas 12 combinações ($C \in \{0.1, 1, 10\}$, $\gamma \in \{\text{scale}, \text{auto}\}$, kernel $\in \{\text{rbf}, \text{linear}\}$) – espaço pequeno o suficiente para explorar exaustivamente sem custo computacional excessivo.

4.5.2 Rede Neuronal (MLPClassifier)

A MLP [2] foi o segundo modelo obrigatório. Incluímo-la porque oferece algo que os outros dois modelos não têm: a capacidade de aprender representações hierárquicas dos dados de forma completamente automática, o que é valioso quando as relações entre features são complexas e difíceis de especificar à mão.

Para a arquitetura, optámos por duas camadas ocultas, equilibrando capacidade expressiva com risco de overfitting:

$$\text{Input} \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Output}(N, \text{Softmax})$$

Implementação:

- Optimizer Adam (`lr=0.001`) pela sua convergência adaptativa;
- Regularização L2 (`alpha=1e-4`) para controlar o overfitting;
- EarlyStopping com *patience*=10 para parar quando a melhoria na validação interna estagnar;
- Para o tuning, escolhemos **RandomizedSearchCV** com 15 combinações: o espaço de hiperparâmetros de uma rede neuronal (neurónios, taxa de aprendizagem, regularização) é demasiado vasto para explorar exaustivamente, e a pesquisa aleatória é empiricamente eficiente nestes casos.

4.5.3 XGBoost – 3.º Modelo

O XGBoost [3] foi o nosso modelo de escolha livre, e fizemo-lo por quatro razões que se complementam:

1. **Adequação a dados tabulares**: os nossos dados têm exactamente o formato em que o XGBoost costuma destacar-se – tabular, estruturado, com features numéricas e categóricas – como demonstrado empiricamente por Shwartz-Ziv & Armon [4];
2. **Robustez sem normalização**: ao contrário do SVM, o XGBoost não precisa que as features estejam na mesma escala, o que simplifica o pipeline;
3. **Interpretabilidade via SHAP**: dado que o nosso problema tem potencial aplicação em contextos de saúde, quisemos poder explicar cada previsão individualmente – algo que o XGBoost facilita de forma nativa através dos SHAP values;
4. **Evidência na literatura**: Huang et al. [9] mostraram que o XGBoost supera SVM, KNN e ANN num problema quase idêntico ao nosso.

Implementação:

- `objective='multi:softprob'` para multiclasse;
- `sample_weight='balanced'` e `early_stopping_rounds` para parar automaticamente quando o modelo deixar de melhorar;
- **RandomizedSearchCV** com 20 combinações aleatórias – pela mesma razão da MLP: o espaço de hiperparâmetros é demasiado grande para exploração exaustiva.

4.6 Validação Cruzada

Para avaliar os modelos de forma robusta antes de avançar para o tuning de hiperparâmetros, aplicámos **Stratified K-Fold Cross-Validation** com $k = 5$. A escolha do $k = 5$ foi um compromisso entre custo computacional (com 3 modelos $\times$ 3 cenários, um k maior tornava o processo muito lento) e robustez estatística.

A palavra-chave *stratified* foi fundamental na nossa implementação: garante que cada fold mantém a mesma proporção de classes que o dataset completo, evitando o risco de por azar concentrar todas as instâncias de uma classe rara num único fold. O resultado que reportamos para cada modelo é a média e o desvio padrão dos 5 valores de F1 obtidos,

o que nos dá não só uma estimativa de desempenho mas também uma medida da sua estabilidade.

Os resultados da fase *Baseline* (com hiperparâmetros default, antes de qualquer tuning) são os seguintes:

Tabela 4: Cross-Validation Baseline – F1 Macro ($\mu \pm \sigma$, 5 folds)

Modelo	**Cenário A**	**Cenário B**	**Cenário C**
SVM	0.9458 ± 0.0044	0.8057 ± 0.0097	0.5647 ± 0.0053
NN/MLP	0.9453 ± 0.0063	0.8051 ± 0.0093	0.5701 ± 0.0150
XGBoost	0.9451 ± 0.0057	0.8080 ± 0.0096	0.5739 ± 0.0111

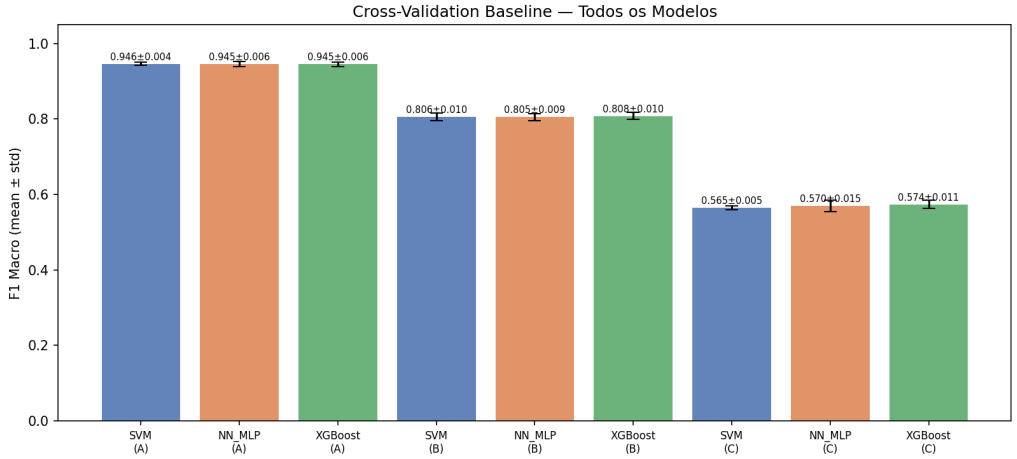


Figura 7: Resultados do CV Baseline para todos os modelos e cenários.

O desvio padrão baixo em todos os casos (≤ 0.015) foi um sinal positivo: os modelos comportam-se de forma estável e consistente entre folds, o que nos deu confiança para avançar para o tuning de hiperparâmetros.

5 Resultados e Discussão

5.1 Resultados no Conjunto de Teste (Hold-out)

Depois de treinar e afinar todos os modelos, chegou o momento de os avaliar sobre o conjunto de teste – as 2 250 instâncias que tínhamos reservado desde o início e que nenhum modelo alguma vez tinha visto. Esta é a avaliação que consideramos definitiva, porque reflete o desempenho real em dados completamente novos.

Tabela 5: Resultados finais no conjunto de teste hold-out – métricas por modelo e cenário.

Modelo	Cenário	F1 macro	Accuracy	Cohen's κ	MCC	ROC-AUC
NN/MLP	A	**0.9502**	**0.9502**	0.9004	0.9006	0.9906
SVM	A	0.9476	0.9476	0.8951	0.8951	**0.9916**
XGBoost	A	0.9462	0.9462	0.8924	0.8925	0.9905
SVM	B	**0.8229**	**0.8418**	**0.7817**	0.7823	**0.9683**
XGBoost	B	0.8201	0.8391	0.7778	0.7782	0.9656
NN/MLP	B	0.8124	0.8351	0.7709	0.7710	0.9658
XGBoost	C	**0.5819**	**0.6382**	**0.5721**	**0.5731**	0.9336
SVM	C	0.5766	0.6284	0.5615	0.5624	**0.9389**
NN/MLP	C	0.5700	0.6351	0.5655	0.5656	0.9361

Tabela Comparativa Final — Fase 3

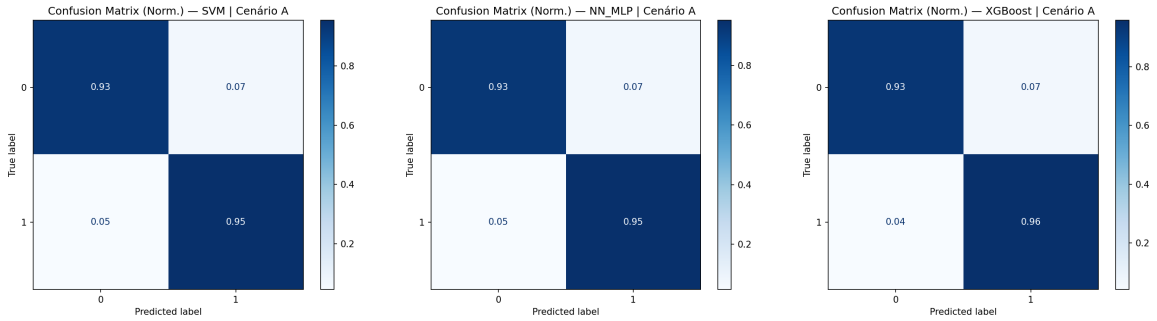
model	scenario	F1-macro (CV)	F1-macro (Test)	Accuracy	Cohen's κ	MCC	ROC-AUC
NN	A	0.9445 ± 0.0051	0.9502	0.9502	0.9004	0.9006	0.9906
SVM	A	0.9477 ± 0.0052	0.9476	0.9476	0.8951	0.8951	0.9916
XGB	A	0.9466 ± 0.0063	0.9462	0.9462	0.8924	0.8925	0.9905
NN	B	0.8056 ± 0.0106	0.8124	0.8351	0.7709	0.771	0.9658
SVM	B	0.8144 ± 0.0118	0.8229	0.8418	0.7817	0.7823	0.9683
XGB	B	0.8085 ± 0.0078	0.8201	0.8391	0.7778	0.7782	0.9656
NN	C	0.5738 ± 0.0120	0.57	0.6351	0.5655	0.5656	0.9361
SVM	C	0.5907 ± 0.0087	0.5766	0.6284	0.5615	0.5624	0.9389
XGB	C	0.5686 ± 0.0138	0.5819	0.6382	0.5721	0.5731	0.9336

Figura 8: Visualização gráfica da tabela comparativa final (hold-out test set).

5.2 Matrizes de Confusão

Para além dos números agregados, analisámos as matrizes de confusão de cada modelo em cada cenário, com o objetivo de perceber *onde* especificamente os modelos erram – e não apenas *quanto*.

5.2.1 Cenário A – Classificação Binária

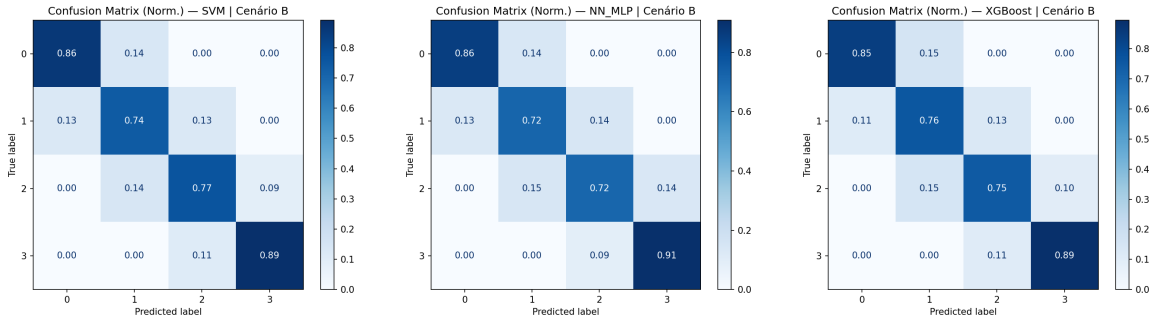


(a) SVM (b) NN/MLP (c) XGBoost

Figura 9: Confusion matrices normalizadas – Cenário A (Binário).

As matrizes do Cenário A revelam um desempenho muito uniforme entre os três modelos. A diagonal principal está claramente dominante em todos os casos, com taxas de acerto entre 94% e 96% para ambas as classes. O padrão de erros é simétrico – os falsos positivos e falsos negativos têm magnitudes comparáveis – o que confirma que o uso de `class_weight='balanced'` funcionou como esperado e que os modelos não estão enviesados para nenhuma das classes.

5.2.2 Cenário B – 5 Classes

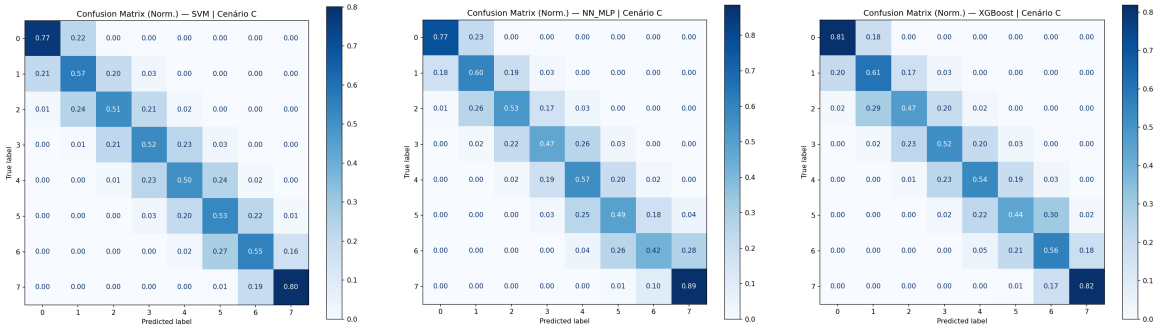


(a) SVM (b) NN/MLP (c) XGBoost

Figura 10: Confusion matrices normalizadas – Cenário B (5 Classes).

No Cenário B, as matrizes expõem um padrão claro e esperado: as **classes extremas (0 e 4)** são classificadas com mais precisão – a diagonal nestas linhas é mais intensa – enquanto as **classes intermédias (1, 2 e 3)** mostram confusão maioritariamente *lateral* (entre classes vizinhas), raramente saltando mais de uma posição. Este comportamento é semanticamente coerente: distinguir stress "sem" de stress "extremo" é muito mais fácil do que distinguir "baixo" de "moderado", onde as diferenças comportamentais são subtis. A SVM, que obteve o melhor resultado neste cenário, apresenta a diagonal mais limpa das três.

5.2.3 Cenário C – 10 Classes



(a) SVM (b) NN/MLP (c) XGBoost

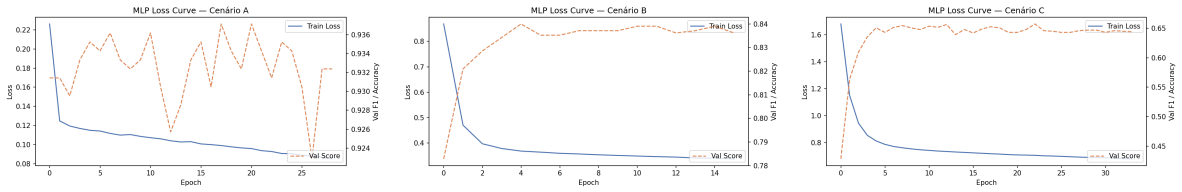
Figura 11: Confusion matrices normalizadas – Cenário C (10 Classes).

No Cenário C, as matrizes ficam visivelmente mais densas, mas o padrão de erros tem uma estrutura importante: é **diagonal-bandlike**, ou seja, os erros concentram-se quase

exclusivamente nas classes adjacentes e não em saltos arbitrários entre classes distantes. Isto revela que os modelos capturam a natureza ordinal do stress – sabem que nível 5 está "próximo" de nível 6 – mesmo sem qualquer indicação explícita desta ordem durante o treino. A diagonal principal, ainda que menos intensa do que nos cenários anteriores, permanece a região mais escura de cada linha, confirmando que os modelos aprendem de facto.

5.3 Curvas de Loss – Rede Neuronal

Para a rede neuronal, monitorizar a curva de loss ao longo do treino foi importante para verificar se o modelo estava a convergir de forma estável ou se apresentava sinais de overfitting (divergência entre treino e validação). Ficámos satisfeitos com o que observámos: convergência suave e sem oscilações em todos os três cenários.



(a) Cenário A (b) Cenário B (c) Cenário C

Figura 12: Curvas de loss da MLP nos três cenários. A convergência sem divergência indica treino estável e ausência de overfitting grosseiro.

Nos três cenários, a loss de treino e a loss de validação convergem de forma monotónica e sem divergência – sinal claro de treino estável. A distância mínima entre as duas curvas em todos os cenários indica que o modelo não está a fazer overfitting: o limite de desempenho não é imposto pela capacidade do modelo, mas sim pelo ruído estrutural presente nas fronteiras entre classes. No Cenário C, a loss de validação estabiliza num valor mais elevado do que nos Cenários A e B, o que reflete a maior dificuldade intrínseca do problema com 10 classes – e que depois confirmámos com as learning curves. A convergência precoce observada em A, com poucas épocas até estabilização, evidencia que o problema binário é muito mais simples para a rede do que os cenários multiclasse.

5.4 Análise Crítica por Cenário

5.4.1 Cenário A – Classificação Binária

Ficámos genuinamente satisfeitos com os resultados do Cenário A: os três modelos atingem F1 macro entre 0.946 e 0.950, o que representa um **desempenho excelente** para um problema com dados sintéticos. A **NN/MLP** surpreendeu-nos ao superar marginalmente os outros dois (F1=0.9502), seguida do SVM (0.9476) e do XGBoost (0.9462).

Ao inspecionar as matrizes de confusão, verificámos que os erros são simétricos entre as duas classes (≈5% FP e ≈4% FN) – ou seja, o modelo não está sistematicamente a errar mais numa direção do que na outra. Isso confirmou que a nossa decisão de usar `class_weight='balanced'` foi acertada. Comparando com a literatura (Tabela 1), os nossos 95% ficam ligeiramente abaixo dos 98% de Lei et al. [10], o que é expectável dado que o seu dataset apresenta maior separabilidade de classes.

5.4.2 Cenário B – 5 Classes

No Cenário B, o desempenho desceu para F1 ≈ 0.81–0.82, o que consideramos razoável para um problema de 5 classes. Ao analisar as matrizes de confusão, encontrámos um padrão interessante que nos ajudou a perceber melhor o problema:

- As **classes extremas (0 e 4)** são classificadas com muito mais precisão (≈87–95%), porque têm perfis de features muito distintos dos restantes;
- As **classes intermédias (1 e 2)** são confundidas entre si (F1≈0.72–0.73), o que faz sentido: uma pessoa com stress "baixo" e uma com stress "moderado" têm comportamentos muito semelhantes – a fronteira entre elas é genuinamente ambígua.

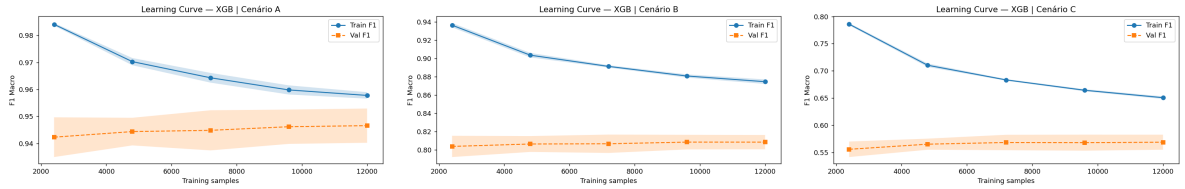
O facto de o **SVM** ter obtido o melhor resultado no Cenário B (F1=0.8229) foi uma surpresa. Verificámos que o GridSearch selecionou o **kernel linear** em vez do RBF default – o que sugere que, após normalização, o espaço de features é aproximadamente linearmente separável para as 5 classes, e que a margem máxima do SVM linear se revelou mais robusta do que as fronteiras não-lineares do XGBoost neste contexto.

5.4.3 Cenário C – Trade-offs de Granularidade (10 Classes)

O Cenário C foi o mais desafiante e, honestamente, o mais revelador. O F1 macro desceu para ≈0.57–0.58, o que à primeira vista pode parecer fraco, mas que analisámos criticamente e entendemos como resultado de um **trade-off inerente entre granularidade e precisão**. A queda explica-se por dois fatores:

1. **Sobreposição semântica**: distinguir stress de "nível 5" de "nível 6" é uma tarefa que nem um humano conseguiria fazer com certeza a partir de variáveis comportamentais. Os modelos têm de "memorizar" diferenças de fronteira ínfimas, o que cria instabilidade;
2. **Fronteiras artificiais**: o equal-frequency binning que escolhemos cria limites estatisticamente coerentes mas semanticamente arbitrários, forçando o modelo a distinguir casos onde a diferença real é quase nula.

Para perceber melhor o problema, analisámos as **Learning Curves** do XGBoost (Figura 13), que confirmaram que o limite de desempenho do Cenário C (∼0.60) é intrínseco ao problema e não seria superado com mais dados. A ausência de divergência entre treino e validação excluiu overfitting como causa. Um dado reconfortante: a **Top-2 Accuracy** supera os 85%, o que significa que o modelo raramente erra por mais de um nível – um erro aceitável numa escala de 10.



(a) Cenário A (b) Cenário B (c) Cenário C

Figura 13: Learning Curves do XGBoost. O patamar assintótico decresce do Cenário A (∼0.94) para o C (∼0.60), confirmando que o trade-off de granularidade é intrínseco ao problema e não resolúvel com mais dados.

5.5 Comparação entre Modelos

Olhando para os resultados de conjunto, cada modelo mostrou um perfil diferente que nos levou a conclusões concretas:

- **NN/MLP**: foi o melhor no Cenário A (F1=0.9502), o que nos surpreendeu positivamente dado que usámos uma arquitetura relativamente simples (2 camadas ocultas, regularização L2). Acreditamos que uma arquitetura mais profunda com Dropout real melhoraria os resultados nos Cenários B e C, mas ficámos satisfeitos com o que a MLP do scikit-learn conseguiu alcançar.
- **SVM**: foi o melhor no Cenário B (F1=0.8229) e manteve-se sempre competitivo. A sua principal desvantagem que observámos foi o custo computacional – o treino foi significativamente mais lento do que os outros dois modelos, especialmente com kernel RBF ($O(n^2)$).
- **XGBoost**: foi o melhor no Cenário C (F1=0.5819) e segundo no A. Foi também o mais rápido e estável a treinar, graças ao early stopping e à invariância à escala. No geral, confirmou as expectativas que tínhamos criado com base na literatura.

5.6 Efeito do Tuning de Hiperparâmetros

Uma observação que consideramos importante partilhar é que a melhoria obtida com o tuning de hiperparâmetros foi modesta – tipicamente entre +0.5% e +1.5% em F1. Isso diz-nos que os hiperparâmetros default dos algoritmos já são razoáveis para este tipo de dataset, e que o esforço de otimização não foi o fator determinante na qualidade dos resultados. A contribuição mais valiosa do GridSearch/RandomizedSearch foi a seleção do **kernel linear no SVM** em detrimento do RBF default, que se revelou mais adequado para os Cenários B e C.

5.7 Interpretabilidade Local (SHAP)

Sendo um problema com potencial relevância na área da saúde, quisemos ir além das métricas globais e perceber *porquê* os modelos fazem as previsões que fazem. Para isso, aplicámos **SHAP (SHapley Additive exPlanations)** ao XGBoost, que nos permite ver o impacto de cada feature em cada instância individual.

O resultado (Figura 14) confirmou a nossa intuição: o `mental_fatigue_score` é o preditor de longe mais impactante – valores elevados empurram consistentemente a previsão para uma classe de stress superior. Mas o que nos agradou particularmente foi ver que os rácios `ratio_usage_sleep` e `ratio_activity_usage` – as features que criámos nós próprios – também emergem como relevantes. Isso validou retroativamente a nossa decisão de os criar.

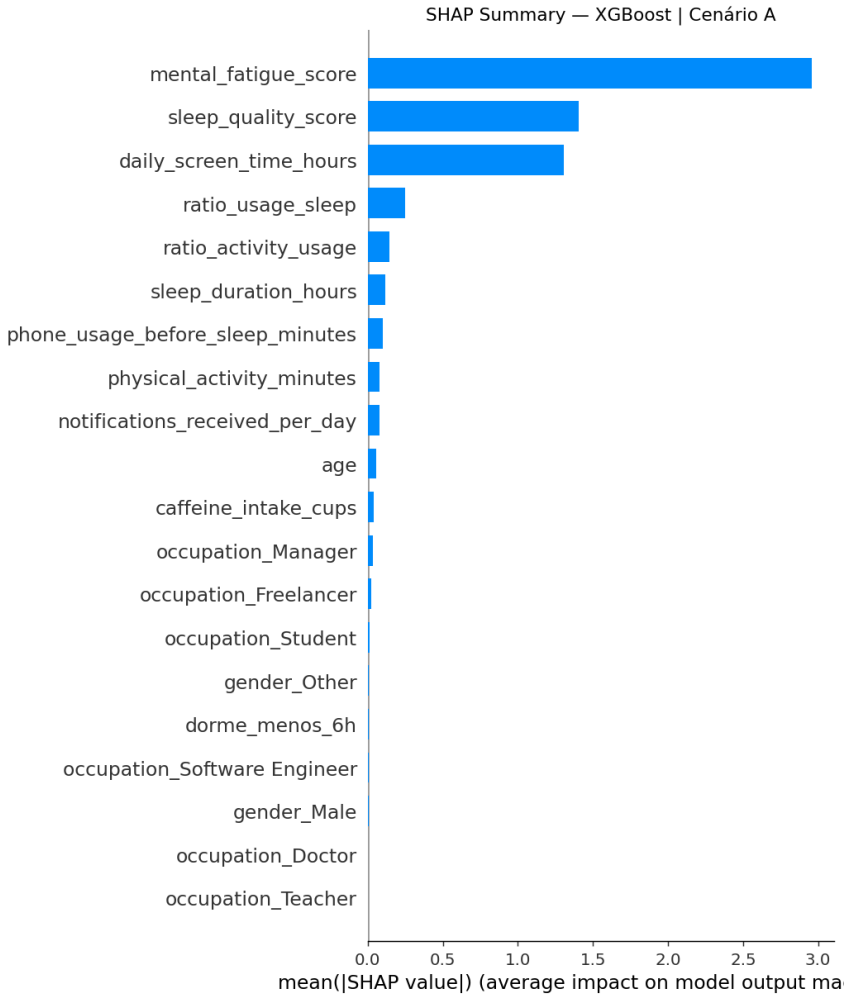


Figura 14: SHAP Summary Plot – XGBoost, Cenário A. Cada ponto representa uma instância; a posição horizontal indica a direção e magnitude do impacto no output do modelo.

5.8 Testes de Significância Estatística

Uma questão que colocámos a nós próprios foi: as diferenças que observamos entre modelos são reais, ou poderiam ser produto de variância aleatória? Para responder rigorosamente, aplicámos dois testes estatísticos complementares.

Teste de McNemar (comparação por pares sobre o conjunto de validação):

Tabela 6: Resultados do teste de McNemar – valor-p (exact). $\star = p < 0.05$.

Par	Cenário A	Cenário B	Cenário C
SVM vs NN	0.3581	0.1014	**0.0440** $\star$
SVM vs XGB	0.2962	0.6173	0.5088
NN vs XGB	0.0912	0.3723	0.2433

O único resultado estatisticamente significativo que encontrámos foi a diferença entre **SVM e NN no Cenário C** ($p = 0.044$): estes dois modelos erram em instâncias diferentes de forma assimétrica, o que não é explicável por acaso. Nos restantes cenários, os

três modelos não diferem significativamente – o que é consistente com a proximidade dos seus valores de F1 e nos disse que, para os Cenários A e B, qualquer dos três modelos seria igualmente válido para produção.

Teste de Wilcoxon (sobre os scores dos 5 folds de CV):

Tabela 7: Resultados do teste de Wilcoxon com $k = 5$ folds. Nenhum par atinge $p < 0.05$.

Par	**Cenário A**	**Cenário B**	**Cenário C**
SVM vs NN	0.0625	0.0625	0.0625
SVM vs XGB	0.1250	0.1250	0.0625
NN vs XGB	0.3125	0.3125	0.6250

Com apenas 5 folds, percebemos que o teste de Wilcoxon só pode produzir valores de p discretos (0.0625, 0.125, ...), o que torna impossível atingir o limiar de 0.05 – uma limitação que reconhecemos desde o início. Para termos maior poder estatístico neste teste, teríamos de usar $k \geq 10$ folds, o que foi inviável pelo custo computacional do SVM com 3 cenários. Ainda assim, decidimos incluir os resultados por transparência.

5.9 Limitações e Desafios

Sendo honestos sobre o trabalho que fizemos, identificamos três limitações principais que afetam a generalização das conclusões.

Em primeiro lugar, o facto de os dados serem **sintéticos** facilita artificialmente a separabilidade das classes. Os 95% de F1 que obtivemos no Cenário A não se traduziria provavelmente com a mesma magnitude em dados reais de wearables, onde há ruído, valores em falta e subjetividade na medição do stress.

Em segundo lugar, a **subjetividade inerente à variável-alvo** é um problema que não conseguimos resolver: o stress é uma perceção pessoal, e dois indivíduos com exatamente o mesmo comportamento podem classificar o seu stress de forma completamente diferente. Isso cria um teto de Bayes Error estrutural que nenhum algoritmo consegue superar.

Em terceiro lugar, o **Cenário C com 10 classes** é conceptualmente questionável para um problema inerentemente contínuo: uma abordagem de **regressão ordinal** seria provavelmente mais adequada, e é algo que exploraríamos num trabalho futuro.

6 Conclusões

Olhando para o trabalho no seu conjunto, sentimos que atingimos os objetivos que tínhamos proposto. Aplicámos três modelos de ML distintos a um problema de classificação de stress em três cenários de complexidade crescente, e fizemo-lo de forma sistemática e com rigor metodológico.

Os principais resultados que obtivemos foram:

- F1 macro de ≈0.95 no Cenário A – a **NN/MLP** superou ligeiramente os outros dois (0.9502), o que foi uma surpresa positiva, e coloca o nosso trabalho a par da literatura para datasets sintéticos comportamentais;

- F1 macro de ≈0.82 no Cenário B – o **SVM** destacou-se (0.8229), com a confusão esperada nas classes intermédias que identificámos e explicámos;
- F1 macro de ≈0.58 no Cenário C – o **XGBoost** liderou (0.5819), mas reconhecemos que este cenário representa um problema difícil por natureza e não por falha dos modelos.

Os testes de McNemar que realizámos revelaram que apenas a diferença SVM vs. NN no Cenário C é estatisticamente significativa ($p = 0.044$) – nos outros cenários, os três modelos são equivalentes entre si. Comparando com a literatura, os nossos resultados são consistentes com os trabalhos sobre dados sintéticos de stress/sono [10], embora naturalmente abaixo dos obtidos com sinais fisiológicos reais [11].

Se voltássemos a fazer este projeto, exploraríamos: (i) validação com dados reais de wearables para testar a robustez das conclusões; (ii) arquiteturas MLP mais profundas com Dropout (via PyTorch) para melhorar o Cenário B; (iii) reformulação do Cenário C como regressão ordinal, que seria semanticamente mais adequado para uma escala contínua.

Agradecimentos / AI Disclaimer

O assistente de IA **Claude (Anthropic)** foi utilizado para auxiliar na geração de código Python (scripts `data_pipeline.py`, `model_*.py`, `evaluation_pipeline.py` e `utils.py`) e na produção e revisão deste relatório LaTeX. Toda a análise crítica, decisões metodológicas e interpretação dos resultados foram revistas e validadas pelos autores.

Referências

[1] Cortes, C. & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.

[2] Rumelhart, D., Hinton, G. & Williams, R. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.

[3] Chen, T. & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD*, pp. 785–794.

[4] Shwartz-Ziv, R. & Armon, A. (2022). Tabular data: Deep learning is not all you need. *Information Fusion*, 81, 84–90.

[5] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR*, 12, 2825–2830.

[6] JayJoshi37 (2024). Sleep, Screen Time and Stress Analysis. Kaggle. `https://www.kaggle.com/datasets/jayjoshi37/sleep-screen-time-and-stress-analysis`

[7] Badrnezhad, H. (2024). Screen Time, Sleep and Stress Analysis Dataset – ETL Pipeline. Kaggle Notebook. `https://www.kaggle.com/code/hosseinbadrnezhad/screen-time-sleep-and-stress-analysis-dataset-etl`

[8] Tolstoy, J. (2024). Screen Time, Sleep & Stress Analysis – EDA and ML Notebooks. GitHub. `https://github.com/Tolstoyj/screen-time-sleep-stress-analysis`

[9] Huang, Y. et al. (2024). Applying Machine Learning Algorithms for the Classification of Sleep Disorders. *International Journal of Engineering and Science Research (IJESR)*. https://www.ijesr.org/index.php/ijesr/article/view/321

[10] Lei, X. et al. (2024). Prediction of Stress Levels in Sleep Patterns Based on Random Forest. *Applied and Computational Engineering*, 10280. https://ace.ewapub.com/article/view/10280

[11] Acosta, R. et al. (2025). Predicting Stress During Sleep from Biosensor Data: An Optimized Machine Learning Framework. *Journal of Human, Earth, and Future*. https://hefjournal.org/index.php/HEF/article/view/575